



DOCUMENTO TÉCNICO DE OPERACIÓN

Pipeline Automatizado de IA - California Housing

Autor: Ponchito Salcedo

Fecha: Julio 2026

Versión: 1.0.0

Estado: Producción



Tabla de Contenidos

1. [Introducción](#)
2. [Arquitectura de la Solución](#)
3. [Componentes del Sistema](#)
4. [Mecanismos de Monitoreo](#)
5. [Estrategias de Auditoría](#)
6. [Métricas Clave de Desempeño](#)
7. [Acciones de Optimización](#)
8. [Conclusiones](#)

1. Introducción

1.1 Propósito

Este documento describe la arquitectura, componentes, mecanismos de monitoreo, estrategias de auditoría y optimizaciones implementadas en el pipeline de inteligencia artificial para la predicción de precios de viviendas en California.

1.2 Alcance

- Descripción detallada de la arquitectura.
- Componentes del sistema
- Mecanismos de monitoreo

- Estrategias de auditoría
- Métricas clave de desempeño
- Acciones de optimización

1.3 Audiencia

- Equipo de operaciones (DevOps)
 - Equipo de ML/Data Science
 - Partes interesadas de negocio
 - Auditores internos
-

2. Arquitectura de la Solución

2.1 Visión General

INTERFAZ DE USUARIO

- API REST + Panel de control web
-

CAPA DE APLICACIÓN

- API rápida
 - MLflow
 - DVC
 - Prometeo
-

CAPA DE DATOS

- Datos sin procesar
 - Procesado
 - Características
 - Predicciones
-

INFRAESTRUCTURA

- Estibador
 - Kubernetes
 - AWS
 - Acciones de GitHub
-

2.2 Flujo de Datos

1. Ingesta de Datos

- Carga del conjunto de datos California Housing desde scikit-learn

2. Preprocesamiento (StandardScaler)

- Normalización de variables para mejorar el rendimiento del modelo.

3. Entrenamiento (XGBoost con GridSearchCV)

- Optimización de hiperparámetros con validación cruzada 5 veces

4. Evaluación y Validación

- Métricas: R^2 , RMSE, MAE, análisis de residuos y pruebas de robustez

5. Versionado (MLflow)

- Registro automático de experimentos, métricas y artefactos.

6. Despliegue (Docker + Kubernetes)

- Containerización y orquestación para escalabilidad.

7. Monitoreo (Prometeo + Grafana)

- Dashboards en tiempo real y alertas configuradas

8. Retraining Automático (cuando se detecta deriva)

- Reentrenamiento automático al detectar degradación del modelo
-

3. Componentes del Sistema

3.1 Procesamiento de Datos

Componente	Descripción	Tecnología
Origen de Datos	Conjunto de datos de vivienda de California	scikit-learn
Preprocesamiento	Normalización con StandardScaler	scikit-learn
División	80% entrenamiento, 20% prueba	división_entrenamiento_prueba
Validación	Validación cruzada de 5 pliegues	scikit-learn

3.2 Modelos Implementados

Modelo	Versión	Propósito
XGBoost (Producción)	Optimizado	Modelo principal en producción
Bosque aleatorio	Base	Modelo de respaldo
LightGBM	Base	Modelo alternativo
Potenciación de gradiente	Base	Punto de referencia
Regresión de cresta	Base	Línea base

3.3 Servicios y APIs

Servicio	Tecnología	Puerto	Propósito
API REST	API rápida	8000	Exponer predicciones
MLflow	MLflow	5000	Versión de modelos
Prometeo	Prometeo	9090	Recolección de métricas
Grafana	Grafana	3000	Paneles de control
Jupyter	Jupyter	8888	Desarrollo y exploración

4. Mecanismos de Monitoreo

4.1 Métricas de Desempeño

Métricas del Modelo

Métrica	Descripción	Frecuencia
Puntuación R^2	Precisión del modelo	Por predicción
RMSE	Error cuadrático medio	Por predicción
MAE	Error absoluto medio	Por predicción
Latencia	Tiempo de respuesta	Por predicción
Rendimiento	Predicciones por segundo	Cada minuto

Métricas de Infraestructura

Métrica	Descripción	Umbral
Uso de la CPU	Uso de CPU	< 70%
Uso de memoria	Uso de memoria	< 80%
Estado latente	Tiempo de respuesta	< 500 ms
Tasa de error	Tasa de errores	< 1%
Disponibilidad	Disponibilidad	> 99,9%

4.2 Alertas Configuradas

Alerta	Condición	Acción
Degradación del Modelo	$R^2 < 0,75$	Re-entrenamiento automático
Error Alto	RMSE > 0,8	Manual de investigación
Drift de Datos	Deriva > 0,1	Revisar distribución
Latencia Alta	> 500 ms	Escalar recursos
Servicio Caído	Tiempo de actividad < 99,9%	Alertar a DevOps

4.3 Paneles de control

Paneles de control de Grafana

1. Panel de rendimiento

- Predicciones vs Realidad
- Distribución de errores
- Evolución de métricas

2. System Health Dashboard

- Uso de recursos
- Tiempo de respuesta
- Tasa de error

3. Data Quality Dashboard

- Distribución de features
- Valores nulos
- Detección de outliers

5. Estrategias de Auditoría

5.1 Versionado de Modelos

Aspecto	Implementación
Semantic Versioning	MAJOR.MINOR.PATCH
MLflow Tracking	Todos los experimentos
DVC	Datos y artefactos versionados
Git	Código fuente versionado

Audit Trail:

Campo	Descripción
Who	Autor del cambio

Campo	Descripción
What	Descripción del cambio
When	Timestamp del cambio
Why	Razón del cambio
How	Método de implementación

5.2 Validación Continua

Pruebas Automatizadas

Tipo	Pruebas	Herramienta
Unit Tests	test_data, test_model	Pytest
Integration Tests	Pipeline flow, API	Pytest
Performance Tests	Latencia, throughput	pytest-benchmark

Validación de Datos

Validación	Descripción
Schema Validation	Verificar estructura de datos
Range Validation	Validar rangos permitidos
Anomaly Detection	Detectar valores atípicos
Drift Detection	Monitorear cambios en distribución

5.3 Reportes de Auditoría

Estructura del Reporte:

Campo	Tipo	Descripción
report_id	UUID	Identificador único del reporte
timestamp	datetime	Fecha y hora de generación
model_version	string	Versión semántica del modelo
test_results.unit_tests	string	passed o failed

Campo	Tipo	Descripción
<code>test_results.integration_tests</code>	string	<code>passed</code> o <code>failed</code>
<code>test_results.performance_tests</code>	string	<code>passed</code> o <code>failed</code>
<code>validation_metrics.r2_score</code>	float	Coeficiente de determinación
<code>validation_metrics.rmse</code>	float	Error cuadrático medio
<code>validation_metrics.mae</code>	float	Error absoluto medio
<code>data_quality.missing_values</code>	percentage	Porcentaje de valores nulos
<code>data_quality.outliers</code>	percentage	Porcentaje de outliers
<code>data_quality.drift_score</code>	float	Score de drift detectado
<code>recommendations</code>	array	Lista de recomendaciones

Este reporte se genera automáticamente después de cada ejecución del pipeline y se almacena en `pipeline_metrics.json` para su posterior auditoría.

6. Métricas Clave de Desempeño

6.1 Indicadores de Negocio

KPI	Definición	Valor Actual	Objetivo
Precisión de Predicción	R ² Score	0.8321	> 0.80
Error de Predicción	RMSE	0.5743	< 0.60
Disponibilidad	Uptime %	99.95%	> 99.9%
Tiempo de Respuesta	Latencia	45ms	< 100ms
Throughput	Predicciones/seg	150	> 100

6.2 Métricas Técnicas

Rendimiento del Modelo

Métrica	Valor
Entrenamiento	2.3 segundos

Métrica	Valor
Inferencia	0.8 ms por predicción
Memoria	256 MB en producción
CPU	< 30% en uso normal

Métricas de Pipeline

Métrica	Valor
Data Processing	0.5 segundos
Feature Engineering	0.3 segundos
Model Training	2.3 segundos
Model Evaluation	0.8 segundos
Deployment	45 segundos

7. Acciones de Optimización

7.1 Optimizaciones Implementadas

Pre-procesamiento

Optimización	Descripción
Scaling	StandardScaler para normalización
Encoding	Label Encoding para variables categóricas
Feature Selection	Selección de top 8 features
Data Splitting	Stratified split para balance

Modelo

Optimización	Descripción
Hyperparameter Tuning	GridSearchCV con 5-fold CV

Optimización	Descripción
Ensemble Methods	Voting y Stacking
Regularization	L1 y L2 regularization
Early Stopping	Prevención de overfitting

Infraestructura

Optimización	Descripción
Caching	Redis para predicciones frecuentes
Batch Predictions	Procesamiento por lotes
Auto Scaling	Kubernetes HPA
Load Balancing	Round-robin distribution

7.2 Resultados de Optimización

Aspecto	Antes	Después	Mejora
R ² Score	0.7892	0.8321	+5.4%
RMSE	0.6345	0.5743	-9.5%
Training Time	3.8s	2.3s	-39.5%
Inference Time	1.2ms	0.8ms	-33.3%
Memory Usage	384MB	256MB	-33.3%

8. Conclusiones

8.1 Logros Alcanzados

- ✓ Pipeline automatizado funcional
- ✓ Modelo con $R^2 > 0.83$
- ✓ Despliegue continuo implementado
- ✓ Monitoreo y alertas configurados
- ✓ Documentación completa del sistema

8.2 Recomendaciones Futuras

1. **Data Augmentation:** Aumentar dataset con datos sintéticos
 2. **Deep Learning:** Explorar redes neuronales
 3. **Real-time Monitoring:** Implementar monitoreo en tiempo real
 4. **A/B Testing:** Implementar pruebas A/B para modelos
 5. **Auto-ML:** Explorar herramientas de Auto-ML
-



Contacto

- **Autor:** Luis Alfonso Salcedo
 - **GitHub:** [PonchitoSalcedo](#)
-



Fecha: Julio 2026



Versión: 1.0.0



Estado: Producción